

Domain component inventory

Every class in every domain package, by role folder, with the tables its entities map

Domain	controllers	services	repositories (...Repository)	entities → table	dto	mappers · specs
access M3 · file 03 · 7 tables 4 endpoints	AccessController	AccessOperations, ConfigKeys, ConfigService, OrganisationService	ConfigHistory, Organisation	ConfigHistoryEntry* → <code>zc_config_history</code> Organisation → <code>zc_organisation</code> <i>no entity yet: <code>zc_tenant</code> (JDBC: <code>TenantRegistry</code> (read)), <code>zc_role_assignment</code>, <code>zc_break_glass_grant</code>, <code>zc_user</code>, <code>zc_tenant_whatsapp</code></i>	ConfigEntryView, CreateOrganisationRequest, OrganisationView, SetConfigRequest	AccessMapper
programme M4 · file 05 · 10 tables 18 endpoints	ProgrammeController, ProgrammeVocabularyController	AssessmentTemplateService, ProgrammeOperations, ProgrammeQueryService, ProgrammeService, ReferralTypeService	AssessmentTemplate, AssessmentTemplateVersion, GapRule, GoalType, ObservationType, OutcomeDefinition, Programme, ProgrammeVersion, ReferralType, TaskTemplate	AssessmentTemplate → <code>zc_assessment_template</code> AssessmentTemplateVersion → <code>zc_assessment_template_version</code> GapRule → <code>zc_gap_rule</code> GoalType → <code>zc_goal_type</code> ObservationType → <code>zc_observation_type</code> OutcomeDefinition → <code>zc_outcome_definition</code> Programme → <code>zc_programme</code> ProgrammeVersion → <code>zc_programme_version</code> ReferralType → <code>zc_referral_type</code> TaskTemplate → <code>zc_task_template</code>	AddGoalTypeRequest, AddObservationTypeRequest, AssessmentTemplateSummary, AssessmentTemplateVersionView, CreateAssessmentTemplateRequest, CreateProgrammeRequest, CreateReferralTypeRequest, DefineOutcomeMeasureRequest, GapRuleView, GoalTypeView, ObservationThreshold, ObservationTypeView, OutcomeDefinitionView, ProgrammeDetail, ProgrammeSummary, ProgrammeVersionDetail, ProgrammeVersionState, ProgrammeVersionSummary, ProgrammeVersionView, ReferralTypeView, SetGapRuleRequest, SetTaskTemplateRequest, SetThresholdRequest, TaskTemplateView	ProgrammeMapper, ProgrammesSpecifications
reference M5 · file 04 · 2 tables	—	MemberReferenceService	MemberReference	MemberReference → <code>zc_member_reference</code> <i>no entity yet: <code>zc_external_reference</code></i>	MemberIdentity, MemberView	MemberReferenceMapper
cohort M6 · file 06 · 3 tables 6 endpoints	CohortController	CohortOperations, CohortService, PayerPort, UnconfiguredPayerPort	CohortMembership, Cohort, CohortRun	Cohort → <code>zc_cohort</code> CohortMembership → <code>zc_cohort_membership</code> CohortRun → <code>zc_cohort_run</code>	AddCohortMemberRequest, CohortMembershipView, CohortRunView, CohortView, CreateCohortRequest, InvitableMembership	CohortMapper
enrolment M7 · file 07 · 3 tables 9 endpoints	EnrolmentController	ConsentGateService, EnrolmentOperations, EnrolmentQueryService, EnrolmentService, MemberCareContextService	ConsentRecord, Enrolment	ConsentRecord → <code>zc_consent_record</code> Enrolment → <code>zc_enrolment</code> <i>no entity yet: <code>zc_member_preference</code></i>	ActivateEnrolmentRequest, CaptureConsentRequest, ConsentView, EnrolmentCounts, EnrolmentExitedEvent, EnrolmentView, InviteMemberRequest, MemberCareContext, SuspendEnrolmentRequest, WithdrawEnrolmentRequest	EnrolmentMapper, EnrolmentSpecifications
assessment M8 · file 08 · 3 tables 2 endpoints	AssessmentController	AssessmentCareContext, AssessmentOperations, AssessmentScorer, AssessmentService	Assessment, AssessmentResponse, RiskClassification	Assessment → <code>zc_assessment</code> AssessmentResponse* → <code>zc_assessment_response</code> RiskClassification* → <code>zc_risk_classification</code>	AssessmentResult, AssessmentView, AssignAssessmentRequest, CompleteAssessmentRequest	—
careplan M9 · file 09 · 5 tables 7 endpoints	CarePlanController	CarePlanCareContext, CarePlanOperations, CarePlanQueryService, CarePlanService	CarePlan, Goal, Intervention, PlanReview	CarePlan → <code>zc_care_plan</code> Goal → <code>zc_goal</code> Intervention → <code>zc_intervention</code> PlanReview* → <code>zc_plan_review</code> <i>no entity yet: <code>zc_care_plan_link</code></i>	CarePlanActivatedEvent, CarePlanDecidedEvent, CarePlanSubmittedEvent, CarePlanView, CreateCarePlanRequest, PlanDecisionRequest	CarePlanMapper
carework M10 · file 10 · 4 tables 3 endpoints	TaskController	TaskCareContext, TaskEventListener, TaskOperations, TaskService, TaskTypes	TaskAssignment, TaskDependency, Task	Task → <code>zc_task</code> TaskAssignment* → <code>zc_task_assignment_history</code> TaskDependency → <code>zc_task_dependency</code> <i>no entity yet: <code>zc_work_queue_definition</code></i>	CompleteTaskRequest, CreateTaskRequest, SystemTask, TaskView	TaskMapper, TaskSpecifications
referral M11 · file 11 · 4 tables 5 endpoints	ReferralController	ProviderService, ReferralEventListener, ReferralOperations, ReferralService	ProviderAction, ProviderParticipation, Referral	ProviderAction* → <code>zc_provider_action</code> ProviderParticipation → <code>zc_provider_participation</code> Referral → <code>zc_referral</code> <i>no entity yet: <code>zc_transition</code></i>	CreateReferralRequest, ProviderActionView, ProviderParticipationView, RecordProviderActionRequest, RecordReferralOutcomeRequest, ReferralView, RegisterParticipationRequest	ReferralMapper, ReferralSpecifications
medication M12 · file 12 · 3 tables 2 endpoints	RefillController	MedicationOperations, RefillEventListener, RefillService	MedicationCoordination, RefillRequest	MedicationCoordination → <code>zc_medication_coordination</code> RefillRequest → <code>zc_refill_request</code> <i>no entity yet: <code>zc_adherence_event</code></i>	CreateRefillRequest, RecordFulfilmentRequest, RefillView	RefillMapper
observation M13 · file 13 · 2 tables 3 endpoints	ObservationController	ObservationCareContext, ObservationOperations, ObservationService	Observation	Observation → <code>zc_observation</code> <i>no entity yet: <code>zc_monitoring_schedule</code></i>	CaptureObservationRequest, InvalidateObservationRequest, ObservationResult, ObservationTrend, ObservationView	ObservationMapper
caregap M14 · file 14 · 1 tables 4 endpoints	CareGapController	CareGapOperations, CareGapService, CareGapSupport	CareGap	CareGap → <code>zc_care_gap</code>	CareGapActionRequest, CareGapView, RecordCareGapRequest	CareGapMapper, CareGapSpecifications
engagement M15 · file 15 · 3 tables 1 endpoint	OutreachController	EngagementOperations, OutreachEventListener, OutreachService	OutreachRequest	OutreachRequest → <code>zc_outreach_request</code> <i>no entity yet: <code>zc_outreach_delivery</code>, <code>zc_inbound_message</code></i>	CreateOutreachRequest, OutreachView	OutreachMapper
outcome M16 · file 16 · 2 tables 2 endpoints	OutcomeController	OutcomeOperations, OutcomeService, ProofService	OutcomeObservation, ReportSnapshot	OutcomeObservation → <code>zc_outcome_observation</code> ReportSnapshot* → <code>zc_report_snapshot</code>	OutcomeView, ProgrammeProof, RecordOutcomeRequest	—
audit M17 · file 18 · 2 tables 1 endpoint	AuditController	AuditOperations, AuditQueryService	DomainAudit	DomainAuditRecord* → <code>zc_domain_audit</code> <i>no entity yet: <code>zc_security_event</code> (JDBC: <code>SecurityEventWriter</code>)</i>	AuditQuery, AuditRecordView	AuditMapper, AuditSpecifications
integration M18 · file 19 · 4 tables <i>package-info only</i>	—	—	—	 <i>no entity yet: <code>zc_idempotency_key</code> (JDBC: <code>IdempotencyStore</code>), <code>zc_integration_outbox</code> (JDBC: <code>OutboxWriter</code>), <code>zc_integration_inbox</code>, <code>zc_dead_letter</code></i>	—	—
ai M19 · file 17 · 2 tables <i>package-info only</i>	—	—	—	 <i>no entity yet: <code>zc_ai_recommendation</code>, <code>zc_ai_review</code></i>	—	—

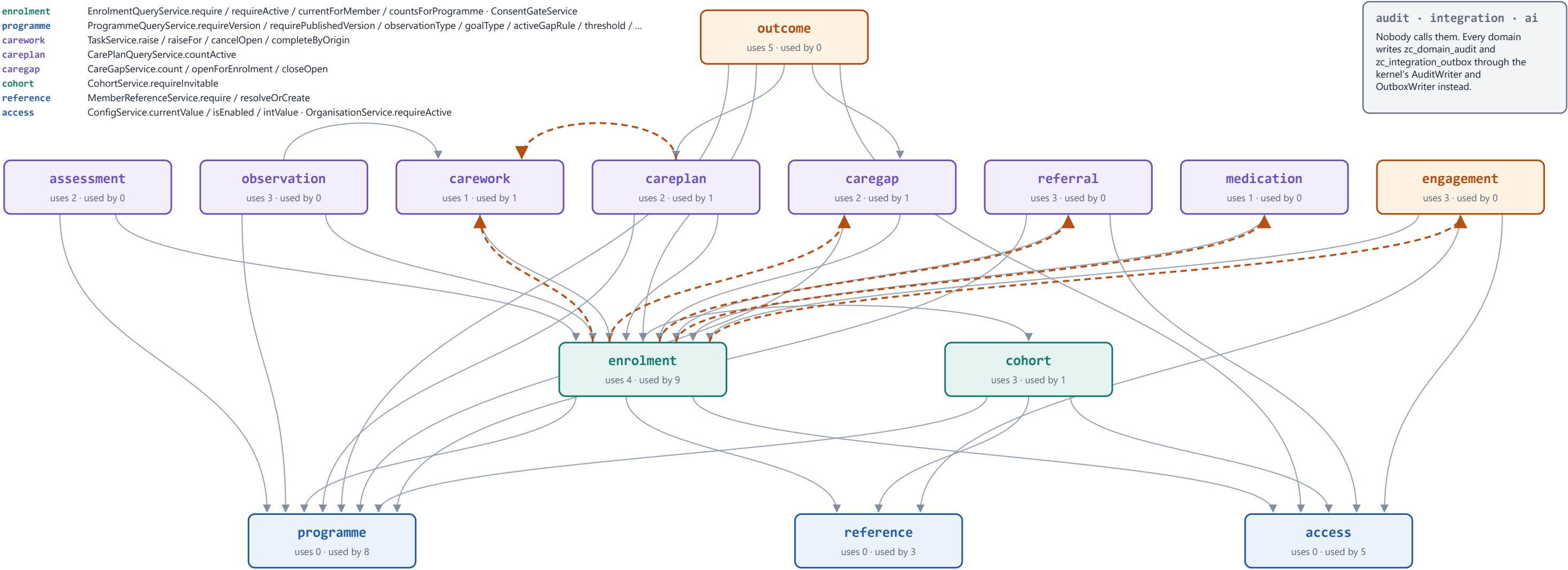
* append-only entity (extends AppendOnlyTenantEntity, @Immutable). Every other entity extends TenantEntity. Services named ...Operations hold audit operation-name constants; ...Listener classes are @EventListener components; ...CareContext and CareGapSupport implement CareContextContributor.

Cross-domain wiring

Synchronous service calls, in-process Spring events and outbox domain events between the 17 domain packages

PROVIDER APIS CALLED ACROSS DOMAINS

enrolment	EnrolmentQueryService.require / requireActive / currentForMember / countsForProgramme · ConsentGateService
programme	ProgrammeQueryService.requireVersion / requirePublishedVersion / observationType / goalType / activeGapRule / threshold / ...
carework	TaskService.raise / raiseFor / cancelOpen / completeByOrigin
careplan	CarePlanQueryService.countActive
caregap	CareGapService.count / openForEnrolment / closeOpen
cohort	CohortService.requireInvisible
reference	MemberReferenceService.require / resolveOrCreate
access	ConfigService.currentValue / isEnabled / intValue · OrganisationService.requireActive



audit · integration · ai
Nobody calls them. Every domain writes `zc_domain_audit` and `zc_integration_outbox` through the kernel's `AuditWriter` and `OutboxWriter` instead.

→ Synchronous call into another domain's service (consumer → provider), same transaction - - - → In-process Spring event (publisher → @EventListener), synchronous in the same transaction Layered: no dependency cycles; enrolment is the hub.

OUTBOX DOMAIN EVENTS BY EMITTING CLASS (OUTBOXWRITER.EMIT, SAME TRANSACTION)

programme	ProgrammeService	ProgrammePublished, ProgrammeRetired
cohort	CohortService	CohortMemberIdentified
enrolment	EnrolmentService	EnrolmentInvited, EnrolmentActivated, EnrolmentSuspended, EnrolmentWithdrawn, ConsentCaptured, ConsentValidated, ConsentRevoked
assessment	AssessmentService	AssessmentCompleted, RiskClassificationChanged
careplan	CarePlanService	CarePlanActivated, CarePlanRevised
carework	TaskService	TaskAssigned, TaskCompleted
referral	ReferralService · ProviderService	ReferralCreated, ReferralAccepted, ReferralCompleted · ProviderActionRecorded
medication	RefillService	RefillRequested, RefillFulfilled
observation	ObservationService	ObservationCaptured, ThresholdReviewRequired
caregap	CareGapService	CareGapDetected, CareGapClosed
engagement	OutreachService	OutreachRequested
outcome	OutcomeService	OutcomeRecorded

28 of the 32 `DomainEventType` values are emitted. Not yet emitted: `TaskOverdue` (needs the SLA sweep), `OutreachDelivered`, `OutreachFailed`, `OutreachCompleted` (need the M15 channel adapter).
Rows land in `zc_integration_outbox` with status pending; nothing drains them yet (integration, M18).

IN-PROCESS EVENTS AND THEIR LISTENERS

CarePlanSubmittedEvent	TaskEventListener.onPlanSubmitted → raise PLAN_APPROVAL (clinician, P2, +72 h)
CarePlanDecidedEvent	TaskEventListener.onPlanDecided → completeByOrigin(care_plan, PLAN_APPROVAL)
CarePlanActivatedEvent	TaskEventListener.onPlanActivated → CARE_PLAN_INTERVENTION task per intervention
EnrolmentExitedEvent	TaskEventListener (cancel open; SAFE_EXIT on member_request) · CareGapSupport (close open) · OutreachEventListener (cancel pending) · RefillEventListener · ReferralEventListener (cancel open)

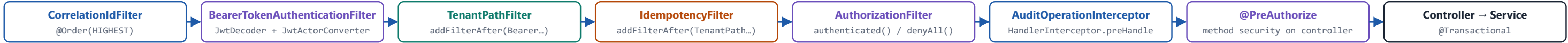
RUNTIME EXTENSION POINTS (NO COMPILE-TIME LINK)

ConsentGate	Implemented by <code>enrolment.ConsentGateService</code> . Called by <code>assessment</code> , <code>careplan</code> , <code>carework</code> , <code>caregap</code> , <code>observation</code> , <code>referral</code> , <code>medication</code> , <code>engagement</code> and <code>outcome</code> services, and by <code>enrolment</code> itself.
CareContextContributor	Five beans (<code>assessment</code> , <code>careplan</code> , <code>carework</code> , <code>observation</code> , <code>caregap</code>) injected as a List into <code>enrolment.MemberCareContextService</code> .
PayerPort	<code>cohort</code> 's port to the payer; the only bean today is <code>UnconfiguredPayerPort</code> .

Platform kernel I: inbound pipeline

Classes that authenticate, bind the tenant, guard idempotency and carry request context

SERVLET & HANDLER ORDER FOR /{TENANT}/ZCARE/V1/**



Request context · common.context, common.web

«@Component @Order(HIGHEST_PRECEDENCE)»
CorrelationIdFilter
common.web

HEADER = "X-Correlation-Id"
SAFE = [A-Za-z0-9._:-]{1,128}

#doFilterInternal(req, res, chain)
Accepts a safe caller id or makes a UUID; sets RequestCorrelation,
MDC and the response header; clears them in finally.

uses → RequestCorrelation, MDC

«final class»
RequestCorrelation
common.context

-ThreadLocal requestId, correlationId
+set(requestId, correlationId)
+requestId() / correlationId()
+clear()

«record»
Actor
common.context

id: String // token sub
roles: Set<ZcareRole>
organisationId: Long // provider users
tenantCode: String

+hasRole(ZcareRole): boolean
+primaryRole(): ZcareRole

«enum»
ZcareRole
common.context

CLINICIAN, CARE_MANAGER, PROGRAMME_ADMIN, PROVIDER_COORDINATOR,
PAYER_MANAGER, EMPLOYER_SPONSOR, PLATFORM_ADMIN, MEMBER

+code() / authority() → ROLE_<NAME>
+fromCode(String): Optional<ZcareRole>
Declaration order is precedence: primaryRole() picks the first held.

«final class»
CurrentActor
common.context

SYSTEM = "system"
+current(): Optional<Actor>
+require(): Actor
+idOrSystem(): String
uses → SecurityContextHolder

«final class»
RequestOperation
common.context

-ThreadLocal<String> CURRENT
+set(op) / clear()
+currentOr(fallback): String
Set from the handler's @AuditOperation by AuditOperationInterceptor
(registered in WebConfig), so side-effect writes are audited under
the causing call.

Authentication · security

«@Configuration @EnableWebSecurity @EnableMethodSecurity»
SecurityConfig
security

+apiFilterChain(http, jwtDecoder, props, tenants, events,
idempotency, json): SecurityFilterChain

Stateless; CSRF, basic, form login and logout off.
permitAll: /error, /actuator/health/**, /actuator/info,
/v3/api-docs/**, /swagger-ui/**
authenticated: /*/zcare/v1/** anyRequest: denyAll
401 entry point logs AUTHENTICATION_FAILED when a token was sent; 403
handler logs AUTHORISATION_DENIED.

uses → JwtActorConverter, TenantPathFilter, IdempotencyFilter,
SecurityEventWriter

«@ConfigurationProperties("zcare.security") record»
ZcareSecurityProperties
security

jwt.issuerUri, jwt.jwkSetUri
jwt.audience = "zimasa-zcare-service"
jwt.hmacSecret // local runs & tests only
claims.tenant = "tenant", claims.organisation = "org"

«@Configuration»
JwtDecoderConfig
security

MIN_HMAC_SECRET_BYTES = 32
+jwtDecoder(props): JwtDecoder

Key source: hmacSecret (HS256) › jwkSetUri › issuerUri; none set →
the service refuses to start.
Validators: timestamp, issuer, audience, sub + exp required.

«Converter<Jwt, AbstractAuthenticationToken>»
JwtActorConverter
security

+convert(jwt): ActorAuthenticationToken
-roles(jwt): role, roles, realm_access.roles,
resource_access.<aud>.roles
-organisation(jwt): Long
ActorAuthenticationToken(principal Actor, credentials Jwt,
authorities ROLE_*)
Unknown role codes are ignored; a token with no ZCare role
authenticates but every endpoint refuses it.

«@RestControllerAdvice @Order(HIGHEST)»
SecurityExceptionAdvice
security

+accessDenied(AccessDeniedException) → 403
+unregisteredRoute(NoResource | NoHandler | MethodNotSupported) →
403
Both write AUTHORISATION_DENIED. Services throw AccessDeniedException
for author self-approval, foreign queues and other organisations'
referrals.
uses → SecurityEventWriter, SecurityResponses

Tenant binding · security, tenant

«OncePerRequestFilter»
TenantPathFilter
security

TENANT_HEADER = "X-Tenant-Id"
API_PATH = ^/([^\/]+)/zcare/v1(/.*)?>\$

#shouldNotFilter(req) // non-API paths
#doFilterInternal(req, res, chain)
No actor: pass on, authorisation answers 401.
Path or header ≠ token tenant: 403 tenant_mismatch +
TENANT_VIOLATION_ATTEMPT.
Tenant not active: 403 tenant_not_active.
TenantContext.set(tenant) ... finally clear().
uses → CurrentActor, TenantRegistry, TenantContext, SecurityEventWriter

«@Component»
TenantRegistry
tenant

CACHE_TTL = 60 s
-cache: ConcurrentHashMap<String, Cached>
+findActive(code): Optional<CurrentTenant>
SELECT id, code FROM zc_tenant WHERE code = ? AND status = 'active'.
zc_tenant has no RLS, so this runs before a tenant is set.
uses → JdbcTemplate

«record»
CurrentTenant
tenant

id: long
code: String

«final class»
TenantContext
tenant

-ThreadLocal<CurrentTenant> CURRENT
+set(t) / current() / clear()
+require(): CurrentTenant // throws when unset

«@Configuration»
TenantTransactionConfig
tenant

+transactionManager(emf): PlatformTransactionManager
Replaces Boot's JPA transaction manager.

«JpaTransactionManager»
TenantAwareTransactionManager
tenant

#doBegin(transaction, definition)
After super.doBegin: SELECT set_config('zcare.tenant_id', ?, true) on
the transaction's connection.
Transaction-local: discarded at commit or rollback, so pooled
connections never carry a tenant.
No tenant in context: nothing set, every tenant table reads empty.
uses → TenantContext, Hibernate Session.doWork

Idempotency & security events

«OncePerRequestFilter»
IdempotencyFilter
common.idempotency

HEADER = "Idempotency-Key"
REPLAY_HEADER = "Idempotency-Replay"
MUTATING = POST, PUT, PATCH, DELETE

#doFilterInternal(req, res, chain)
Missing key: VALIDATION_FIELD_REQUIRED. Not a UUID:
VALIDATION_FIELD_INVALID.
hash = sha256(body); store.find(key, actor, route).
Hit, same hash: replay the stored envelope with Idempotency-Replay:
true.
Hit, other hash: IDEMPOTENCY_KEY_CONFLICT (HARD_STOP).
Miss: run the chain; cache the reply only if it is an HTTP 200
envelope (business exceptions included).
uses → IdempotencyStore, CachedBodyRequest, ContentCachingResponseWrapper

«HttpServletRequestWrapper»
CachedBodyRequest
common.idempotency

Replays the already-read body bytes to the controller.

«@Component»
IdempotencyStore
common.idempotency

+find(key, actorId, route): Optional<CachedResponse>
+save(key, actorId, route, bodyHash, envelopeJson)
record CachedResponse(bodyHash, envelopeJson)

Each call in its own TransactionTemplate, tenant set.
INSERT ... ON CONFLICT uq_zc_idempotency_key DO UPDATE only when
expired; expires_at = now() + 24 h. Never deletes (zc_app has no
DELETE).
uses → JdbcTemplate → zc_idempotency_key

«@Component»
SecurityEventWriter
security

MAX_DETAIL = 1000
+record(type, tenantId, actorId, request, detail)
TransactionTemplate REQUIRES_NEW → zc_security_event, so the event
survives the request's rollback.
Best effort: a failure is logged, never turns a 403 into a 500.

«enum»
SecurityEventType
security

AUTHORISATION_DENIED, TENANT_VIOLATION_ATTEMPT,
AUTHENTICATION_FAILED, AUTHENTICATION_SUCCEEDED,
INVALID_WEBHOOK_SIGNATURE, BREAK_GLASS_USE, AUDIT_EXPORT,
ACCOUNT_LOCKED

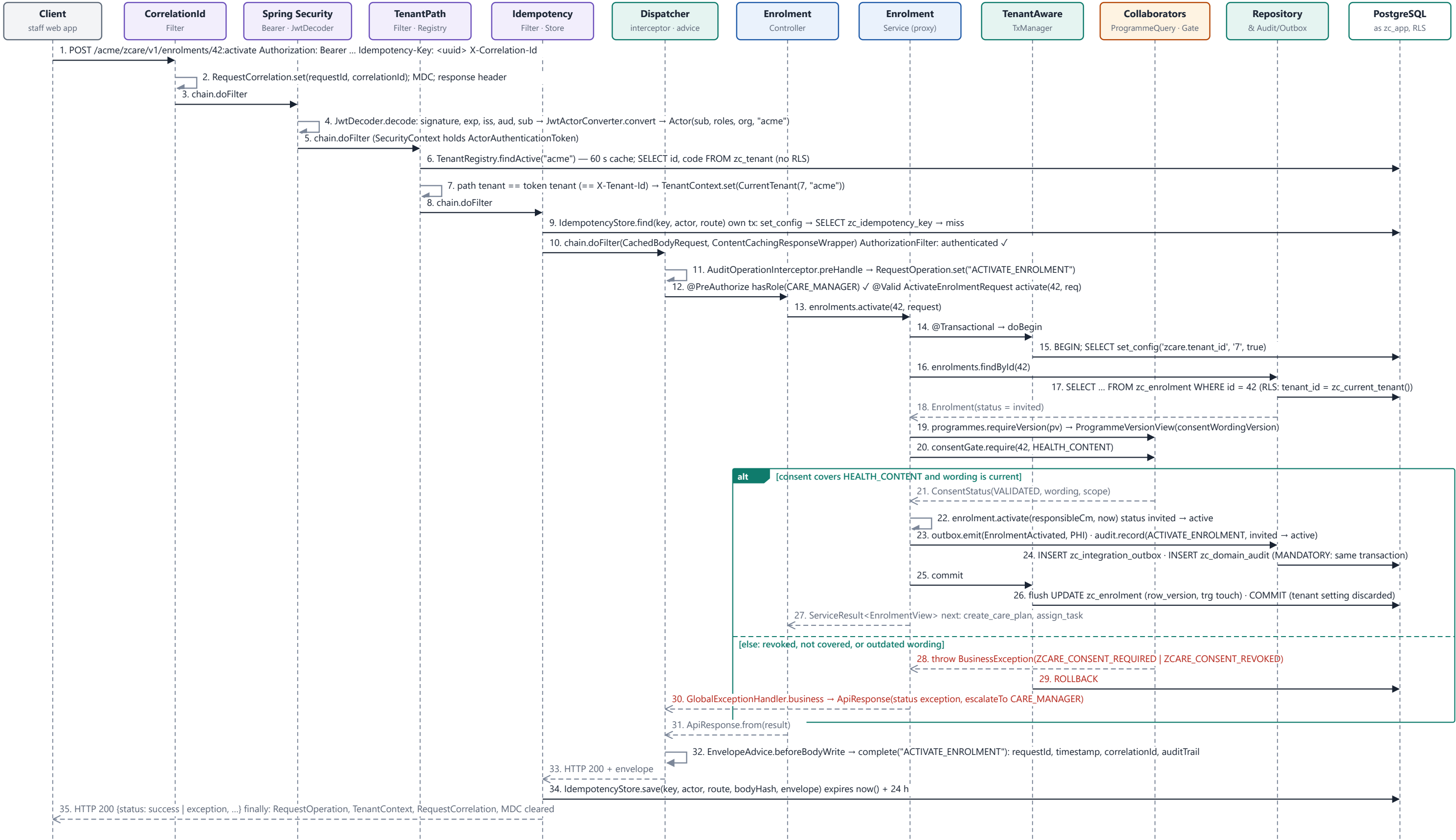
Platform kernel II: contract, errors, persistence

The envelope, the error model, entity base classes, audit and outbox writers, and the extension points domains implement

API contract · common.api, config	Error model · common.error	Persistence & side-effect writers	Extension points · implemented by domains
«@interface @Target(METHOD)» AuditOperation common.api value(): String On every endpoint. Fills auditTrail.operation and must equal the operation the service writes to zc_domain_audit.	«RuntimeException» BusinessException common.error code: ZCareExceptionCode field, context, session, data, nextActions +of(code, message) +invalid(field, msg) → VALIDATION_FIELD_INVALID +required(field, msg) → VALIDATION_FIELD_REQUIRED +unknown(field, what) // id not visible to tenant +with(k, v) / data(o) / session(k, v) / nextActions(...) No stack trace. Thrown from services: rolls the transaction back.	«@MappedSuperclass abstract» TenantEntity common.persistence id: Long @Id IDENTITY tenantId: Long updatable = false createdAt, createdBy updatable = false updatedBy: String rowVersion: Integer = 1 @Version @PrePersist beforeInsert(): tenantId ← TenantContext, createdBy ← CurrentActor @PreUpdate beforeUpdate(): updatedBy Extended by 31 entities. zc_touch_row() bumps row_version and refuses id, tenant and created_* changes.	«interface» ConsentGate common.consent +status(enrolmentId): ConsentStatus +require(enrolmentId, ContentClass): ConsentStatus REVOKED → ZCARE_CONSENT_REVOKED (HARD_STOP); not covered → ZCARE_CONSENT_REQUIRED (ESCALATE).
«final class» ApiResponse<T> common.api requestId, timestamp status: EnvelopeStatus data: T exceptions: List<ApiException> nextActions: List<String> sessionContext: Map<String, Object> auditTrail: AuditContext +success(data) / of(status, data) +from(ServiceResult<T>) // warnings → exception +addException(e) // success → exception +nextActions(...) / session(k, v) +complete(operation)	«enum (24 codes)» ZCareExceptionCode common.error severity: Severity escalateTo: String nextActions: List<String> // empty for HARD_STOP 21 founding codes from 04B plus 3 pending (OD-17). Never renamed or reused. Full registry on the error-model page.	«@MappedSuperclass abstract» AppendOnlyTenantEntity common.persistence id, tenantId, createdAt, createdBy @PrePersist beforeInsert() 8 @Immutable subclasses: ConfigHistoryEntry, AssessmentResponse, RiskClassification, PlanReview, TaskAssignment, ProviderAction, ReportSnapshot, DomainAuditRecord.	«@Service implements ConsentGate» ConsentGateService domains.enrolment.services IN_FORCE = captured, validated, scope_reduced The in-force record decides; else the latest record; else NONE.
«final class» ServiceResult<T> common.api data: T, nextActions, session, warnings +of(data) / next(actions...) / session(k, v) +warning(code, message, context) What every mutating service returns. A warning commits the work but the envelope reports status exception (e.g. outreach held).	«enums» Severity · EnvelopeStatus common.api Severity: INFO, WARNING, ESCALATE, HARD_STOP EnvelopeStatus: success, exception, error, pending	«@Component» AuditWriter common.audit +record(AuditEntry) @Transactional(MANDATORY) JDBC INSERT zc_domain_audit: tenant, actor, organisation, member, programme version, entity, operation, previous/new state (jsonb), reason, correlation, consent wording & channel.	«record + enums» ConsentStatus common.consent (enrolmentId, state, wordingVersion, scope) +covers(cc): state ∈ {VALIDATED, SCOPE_REduced} ∧ cc ∈ scope ConsentState: NONE, CAPTURED, VALIDATED, SCOPE_REduced, REVOKED, SUPERSEDED ContentClass: CONDITION_NEUTRAL, HEALTH_CONTENT
«@RestControllerAdvice ResponseBodyAdvice» EnvelopeAdvice common.api +beforeBodyWrite(body, ...) envelope.complete(@AuditOperation value or "UNSPECIFIED")	«@RestControllerAdvice @Order(Lowest)» GlobalExceptionHandler common.error +business(BusinessException) → 200 exception +invalidBody / invalidParameters / invalidConstraints → 200, one entry per field, [retry] +unreadableBody / typeMismatch / missingParameter / missingHeader → 200 +integrity(DataIntegrityViolation) → translate +concurrent(OptimisticLockingFailure) → ZCARE_CONCURRENT_MODIFICATION +unexpected(Exception) → 500 error, ZCARE_INTERNAL_ERROR Transport errors (415, 406...) keep Spring's status outside the envelope.	«builder» AuditEntry common.audit +of(operation, entityType, entityId) .member(id) .programmeVersion(id) .organisation(id) .previousState(map) .newState(map) .reason(s) .consent(wordingVersion, channel)	«interface» CareContextContributor common.carecontext +section(): String +contribute(enrolmentId): Object AssessmentCareContext "assessment" CarePlanCareContext "carePlan" TaskCareContext "openTasks" ObservationCareContext "recentObservations" CareGapSupport "openCareGaps"
«records» ApiException · AuditContext · Paging common.api ApiException(code, severity, message, field, escalateTo, context) AuditContext(operation, userId, correlationId) PagedResponse<T>(items, count, page, pageSize, totalItems, totalPages, hasNext, ...) PageParams(page, pageSize) DEFAULT 25, MAX 200 CollectionResponse<T>(items, count) ApiPaths.BASE =("/{tenant}/zcare/v1"	«final class» ConstraintViolationTranslator common.error -CODES: Map<constraint, code> // 11 entries +constraintName(Throwable): Optional<String> +translate(Throwable): BusinessException Reads SQLException.getServerErrorMessage().getConstraint() through the cause chain; unmapped → VALIDATION_FIELD_INVALID.	«@Component» OutboxWriter common.outbox +emit(type, aggregateType, aggregateId, payload, classification) @Transactional(MANDATORY) JDBC INSERT zc_integration_outbox (status 'pending'). The M18 drain publishes; not built yet.	«@Service» MemberCareContextService domains.enrolment.services (..., List<CareContextContributor> contributors) +read(memberId, enrolmentId): ServiceResult<MemberCareContext> Audits every read; adds health sections only when consent covers HEALTH_CONTENT.
		«@Component» DomainEventType · DataClassification common.outbox DomainEventType: 32 AsyncAPI event names DataClassification: PHI("phi"), NON_PHI("non_phi")	«interface» PayerPort domains.cohort.services +identify(programmeVersionId, ruleReference): Identification Identification(members, sourceDataDate) PayerUnavailableException
			«@Component implements PayerPort» UnconfiguredPayerPort domains.cohort.services Always throws PayerUnavailableException until a payer adapter exists, so a run is recorded as failed, never as "found nobody".

Sequence: one mutating request end to end

POST /{tenant}/zcare/v1/enrolments/{id}:activate, from the first filter to the committed envelope



Early exits (never reach the controller)

Step 4 Bad, expired or missing token → 401 + WWW-Authenticate; AUTHENTICATION_FAILED event when a token was sent.

Step 9 Same key + same body → stored envelope replayed (Idempotency-Replay: true); different body → IDEMPOTENCY_KEY_CONFLICT.

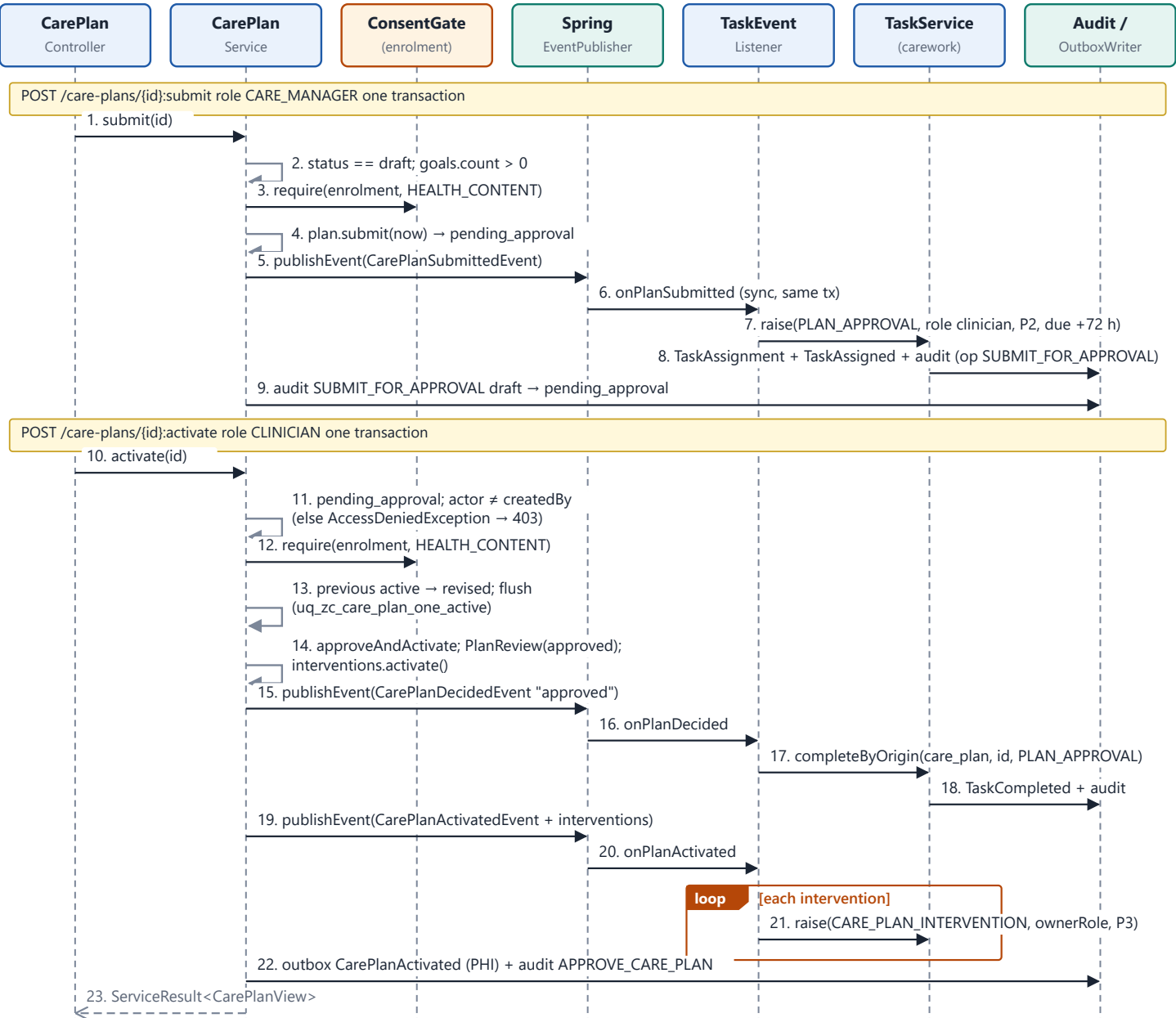
Step 7 Tenant mismatch → 403 tenant_mismatch + TENANT_VIOLATION_ATTEMPT; inactive tenant → 403 tenant_not_active.

Step 12 Role not permitted → AccessDeniedException → SecurityExceptionAdvice → 403 forbidden + AUTHORISATION_DENIED.

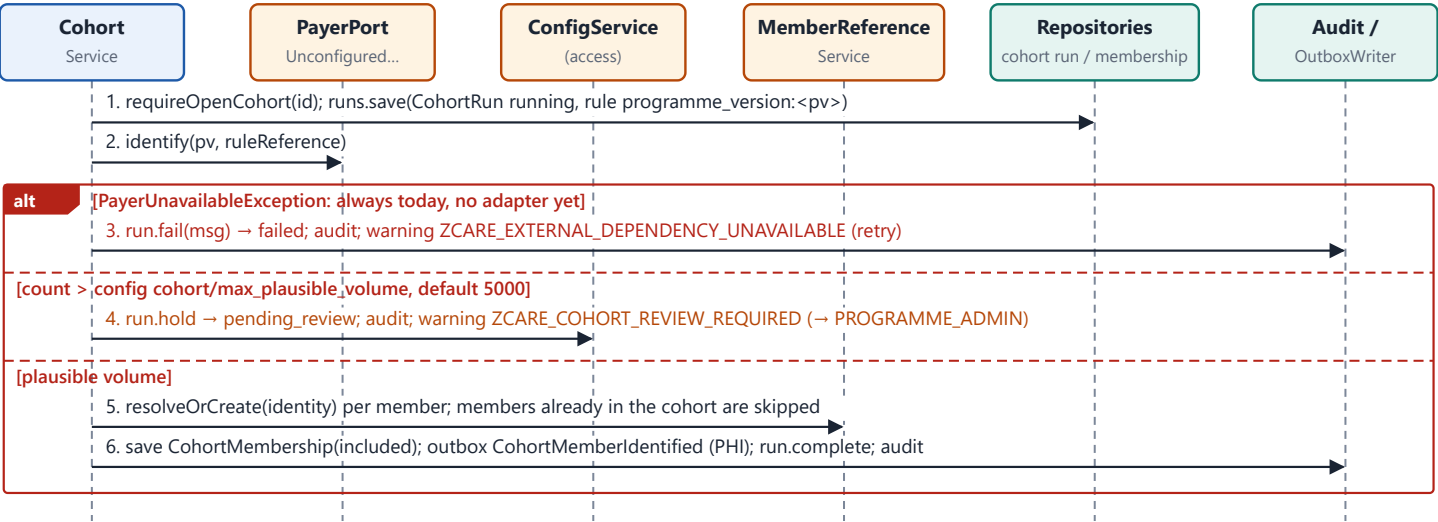
Sequences: event-driven side effects

How one domain’s decision creates or closes another domain’s work inside the same transaction, and how cohort identification degrades

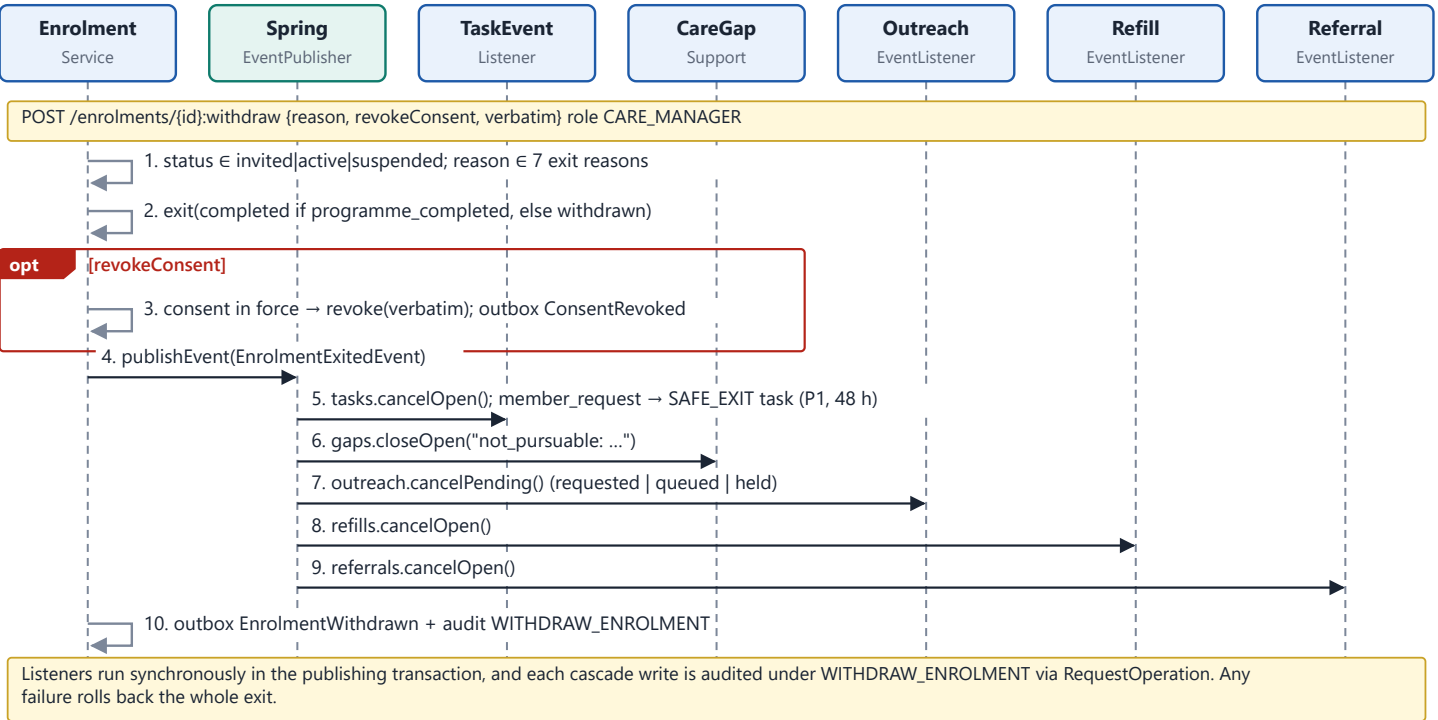
A · CARE PLAN SUBMITTED, THEN APPROVED BY A CLINICIAN



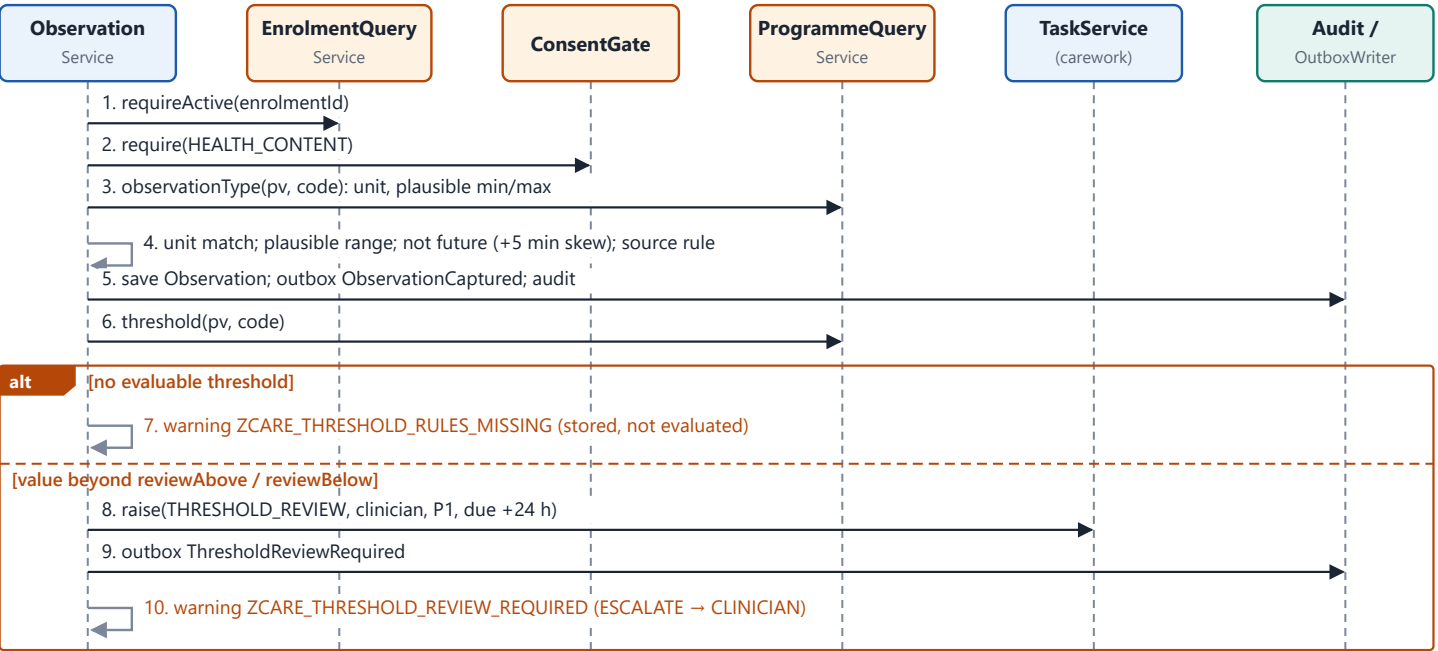
D · COHORT IDENTIFICATION THROUGH THE PAYERPORT (POST /COHORTS/{ID}:IDENTIFY, PROGRAMME_ADMIN)



B · ENROLMENT WITHDRAWN: NO OPEN WORK OUTLIVES IT

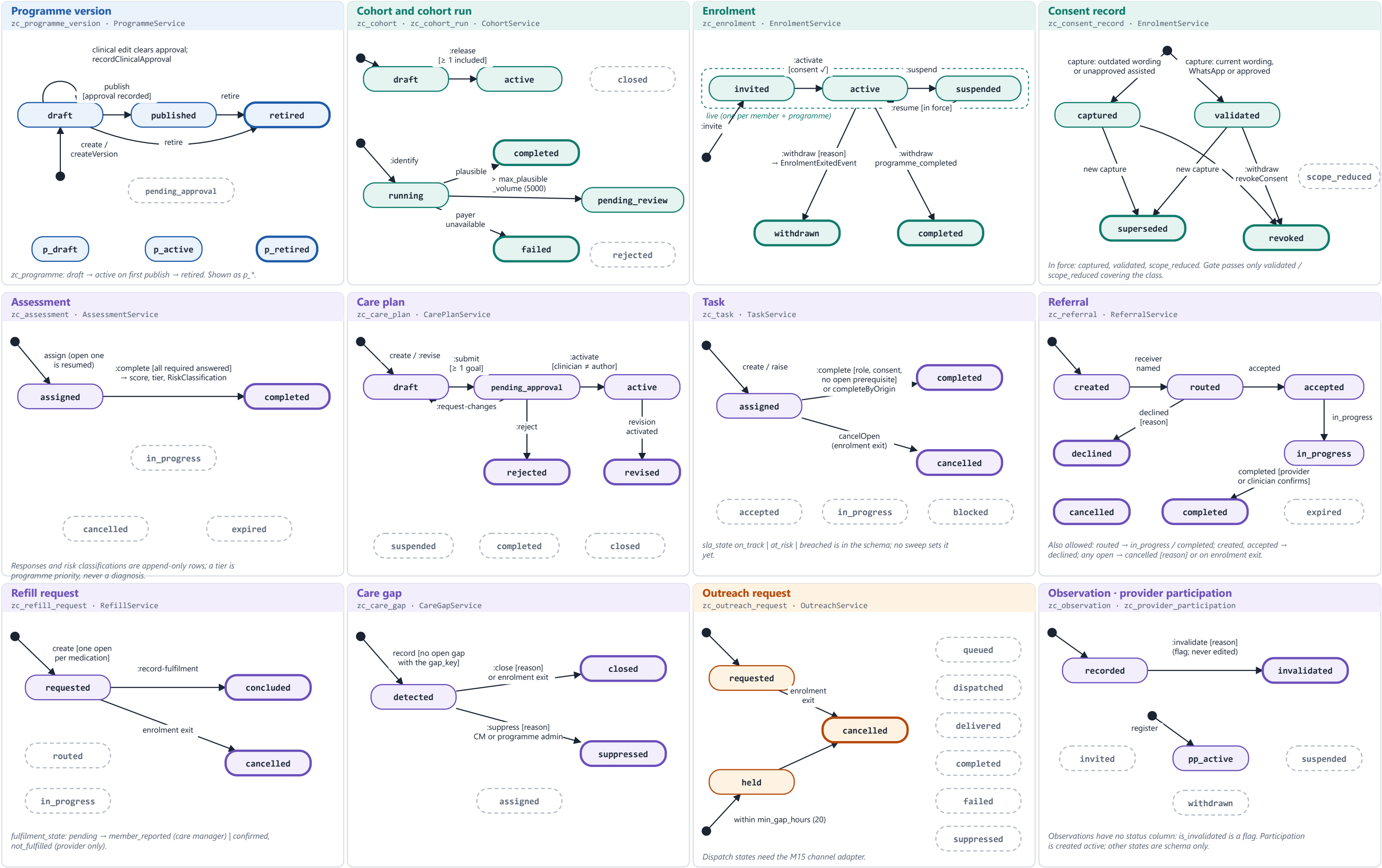


C · OBSERVATION BEYOND THRESHOLD RAISES A CLINICIAN REVIEW



Lifecycle state machines

Status values from the schema CHECK constraints; transitions as the services implement them today



Data model I: foundation, population & consent

Tables of the access, reference, programme, cohort and enrolment migrations, with every business column

Every table also has tenant_id (FK → zc_tenant, leads every index), created_at and created_by; mutable tables add updated_at, updated_by and row_version. Bold = NOT NULL. → = foreign key target (zc_ prefix dropped); FKs are composite with tenant_id, and often programme_version_id or enrolment_id, so a row can only reference rows of the same tenant (and version). Header line 2: the entity class that maps the table, or its JDBC writer.

access · migration file 03

zc_tenant <i>JDBC read: TenantRegistry</i>
id int8 PK
code text
name text
status text

zc_organisation
Organisation

id int8 PK
code text
name text
org_type text
status text

zc_role_assignment
no entity yet

id int8 PK
organisation_id int8 → organisation
actor_id text
role_code text
valid_from tstz
valid_to tstz
granted_by text
revoked_at tstz
revoked_by text
revoke_reason text

zc_break_glass_grant
no entity yet

id int8 PK
actor_id text
reason text
granted_at tstz
expires_at tstz
revoked_at tstz
revoked_by text

zc_config_history
ConfigHistoryEntry

append-only

id int8 PK
config_scope text
config_key text
config_value text
effective_from tstz
change_reason text
approved_by text
approved_at tstz

zc_user
no entity yet

id int8 PK
org int8 → organisation
email text
password_hash text
role text
status text
display_name text
failed_login_count int4
locked_until tstz
last_login_at tstz
password_changed_at tstz

zc_tenant_whatsapp
no entity yet

phone_number_id text
access_token text
active bool

reference · migration file 04

zc_member_reference
MemberReference

id int8 PK
source_system text
source_member_number text
source_individual_ref text
display_name text
contact_msisdn text
source_date date
is_authoritative bool

zc_external_reference
no entity yet

id int8 PK
entity_type text
entity_id int8
system_code text
code_system text
external_id text
source_date date
is_authoritative bool

programme · migration file 05

zc_programme
Programme

id int8 PK
code text
name text
care_model text
status text
description text

zc_programme_version
ProgrammeVersion

id int8 PK
programme_id int8 → programme
version_number int4
status text
content jsonb
clinical_approved_by text
clinical_approved_at tstz
published_at tstz
retired_at tstz

zc_assessment_template
AssessmentTemplate

id int8 PK
code text
name text
description text

zc_assessment_template_version
AssessmentTemplateVersion

id int8 PK
template_id int8 → assessment_template
version_number int4
status text
questionnaire jsonb
scoring_rules jsonb
published_at tstz
retired_at tstz

zc_gap_rule
GapRule

id int8 PK
programme_version_id int8 → programme_version
gap_type text
rule_version int4
period_strategy text
definition jsonb
description text
is_active bool

zc_task_template
TaskTemplate

id int8 PK
programme_version_id int8 → programme_version
task_type text
name text
description text
default_priority int2
sla_hours int4
default_assignee_role text

zc_referral_type
ReferralType

id int8 PK
code text
name text
is_platform_defined bool
is_active bool

zc_observation_type
ObservationType

id int8 PK
programme_version_id int8 → programme_version
code text
name text
unit text
loinc_code text
plausible_min numeric
plausible_max numeric

zc_goal_type
GoalType

id int8 PK
programme_version_id int8 → programme_version
code text
name text

zc_outcome_definition
OutcomeDefinition

id int8 PK
programme_version_id int8 → programme_version
measure_code text
measure_version int4
name text
category text
definition jsonb

cohort · migration file 06

zc_cohort
Cohort

id int8 PK
programme_version_id int8 → programme_version
code text
name text
status text
description text

zc_cohort_run
CohortRun

id int8 PK
cohort_id int8 → cohort
status text
rule_reference text
started_at tstz
finished_at tstz
source_data_date date
result_count int4
hold_reason text
reviewed_by text
reviewed_at tstz
failure_reason text

zc_cohort_membership
CohortMembership

id int8 PK
cohort_id int8 → cohort, cohort_run
member_id int8 → member_reference
cohort_run_id int8 → cohort_run
inclusion_method text
inclusion_source_detail text
inclusion_reason text
source_data_date date
status text
removed_at tstz
removed_by text
removal_reason text

enrolment · migration file 07

zc_enrolment
Enrolment

id int8 PK
member_id int8 → member_reference, cohort_membership, enrolment
programme_id int8 → programme, programme_version
programme_version_id int8 → programme_version
cohort_membership_id int8 → cohort_membership
supersedes_enrolment_id int8 → enrolment
status text
invited_at tstz
activated_at tstz
responsible_cm text
exited_at tstz
exit_reason text
eligibility_snapshot jsonb
eligibility_source_date date

zc_consent_record
ConsentRecord

id int8 PK
enrolment_id int8 → enrolment
member_id int8 → enrolment, member_reference
status text
wording_version text
channel text
scope_content_classes text[]
scope_sharing_purposes text[]
captured_at tstz
captured_by text
evidence_ref text
validated_by text
validated_at tstz
revoked_at tstz
revocation_channel text
revocation_verbatim text
superseded_at tstz

zc_member_preference
no entity yet

id int8 PK
member_id int8 → member_reference
do_not_contact bool
do_not_contact_reason text
do_not_contact_at tstz
preferred_channel text
preferred_language text
contact_window_start time
contact_window_end time

Data model II: care delivery, engagement & platform

Tables of the remaining migrations, with every business column

Every table also has tenant_id (FK → zc_tenant, leads every index), created_at and created_by; mutable tables add updated_at, updated_by and row_version. Bold = NOT NULL. → = foreign key target (zc_ prefix dropped); FKs are composite with tenant_id, and often programme_version_id or enrolment_id, so a row can only reference rows of the same tenant (and version). Header line 2: the entity class that maps the table, or its JDBC writer.

assessment · migration file 08 zc_assessment Assessment id int8 PK enrolment_id int8 → enrolment template_version_id int8 → assessment_template_version status text assigned_at tstz due_at tstz completed_at tstz score numeric score_detail jsonb cancel_reason text	zc_intervention Intervention id int8 PK care_plan_id int8 → care_plan, goal goal_id int8 → goal owner_role text description text frequency text status text status_reason text
---	--

API catalogue

All 67 endpoints under /{tenant}/zcare/v1, with roles, audit operation and handler

Domain		Path under /{tenant}/zcare/v1	Roles	@AuditOperation	Handler	Consent gate	Emits (outbox · in-process)
access	GET	/organisations	PA · PLA	VIEW_ORGANISATIONS	Access·listOrganisations		
	POST	/organisations	PLA	CREATE_ORGANISATION	Access·createOrganisation		
	GET	/config	PLA	VIEW_CONFIG	Access·listConfig		
	POST	/config	PLA	SET_CONFIG	Access·setConfig		
programme	GET	/programmes	PA · CLN	VIEW_PROGRAMMES	Programme·list		
	GET	/programmes/{programmeId}	PA · CLN	VIEW_PROGRAMME	Programme·detail		
	POST	/programmes	PA	CREATE_PROGRAMME	Programme·create		
	POST	/programmes/{programmeId}/versions	PA	CREATE_PROGRAMME_VERSION	Programme·createVersion		
	POST	/programmes/{programmeId}/observation-types	PA	ADD_OBSERVATION_TYPE	Programme·addObservationType		
	POST	/programmes/{programmeId}/thresholds	PA	SET_OBSERVATION_THRESHOLD	Programme·setThreshold		
	POST	/programmes/{programmeId}/goal-types	PA	ADD_GOAL_TYPE	Programme·addGoalType		
	POST	/programmes/{programmeId}/gap-rules	PA	SET_GAP_RULE	Programme·setGapRule		
	POST	/programmes/{programmeId}/task-templates	PA	SET_TASK_TEMPLATE	Programme·setTaskTemplate		
	POST	/programmes/{programmeId}/outcome-measures	PA	DEFINE_OUTCOME_MEASURE	Programme·defineOutcomeMeasure		
	POST	/programmes/{programmeId}:record-clinical-approval	CLN	RECORD_CLINICAL_APPROVAL	Programme·recordClinicalApproval		
	POST	/programmes/{programmeId}:publish	PA	PUBLISH_PROGRAMME	Programme·publish		ProgrammePublished
	POST	/programmes/{programmeId}:retire	PA	RETIRE_PROGRAMME	Programme·retire		ProgrammeRetired
	GET	/referral-types	PA · CM · CLN	VIEW_REFERRAL_TYPES	ProgrammeVocabulary·listReferralTypes		
	POST	/referral-types	PA	CREATE_REFERRAL_TYPE	ProgrammeVocabulary·createReferralType		
	GET	/assessment-templates	PA · CLN · CM	VIEW_ASSESSMENT_TEMPLATES	ProgrammeVocabulary·listAssessmentTemplates		
	POST	/assessment-templates	PA	CREATE_ASSESSMENT_TEMPLATE	ProgrammeVocabulary·createAssessmentTemplate		
	POST	/assessment-templates/{templateId}/publish	PA	PUBLISH_ASSESSMENT_TEMPLATE	ProgrammeVocabulary·publishAssessmentTemplate		
cohort	GET	/cohorts	PA · CM	VIEW_COHORTS	Cohort·list		
	POST	/cohorts	PA	CREATE_COHORT	Cohort·create		
	GET	/cohorts/{cohortId}/members	PA · CM	VIEW_COHORT_MEMBERS	Cohort·members		
	POST	/cohorts/{cohortId}/members	PA · CM	ADD_COHORT_MEMBER	Cohort·addMember		
	POST	/cohorts/{cohortId}:identify	PA	IDENTIFY_COHORT	Cohort·identify		CohortMemberIdentified ×n (or run failed / held)
	POST	/cohorts/{cohortId}:release	PA	RELEASE_COHORT	Cohort·release		
enrolment	GET	/enrolments	CM · CLN · PA	VIEW_ENROLMENTS	Enrolment·list		
	GET	/enrolments/{enrolmentId}	CM · CLN	VIEW_ENROLMENT	Enrolment·get		
	POST	/enrolments:invite	CM	INVITE_MEMBER	Enrolment·invite		EnrolmentInvited
	POST	/enrolments/{enrolmentId}/consent:capture	CM	CAPTURE_CONSENT	Enrolment·captureConsent		ConsentCaptured (+ ConsentValidated)
	POST	/enrolments/{enrolmentId}:activate	CM	ACTIVATE_ENROLMENT	Enrolment·activate	require	EnrolmentActivated
	POST	/enrolments/{enrolmentId}:suspend	CM	SUSPEND_ENROLMENT	Enrolment·suspend		EnrolmentSuspended
	POST	/enrolments/{enrolmentId}:resume	CM	RESUME_ENROLMENT	Enrolment·resume	require	
	POST	/enrolments/{enrolmentId}:withdraw	CM	WITHDRAW_ENROLMENT	Enrolment·withdraw		EnrolmentWithdrawn (+ ConsentRevoked) · EnrolmentExitedEvent
	GET	/members/{memberId}/care-context	CM · CLN	VIEW_MEMBER_CONTEXT	Enrolment·careContext	status	
assessment	POST	/enrolments/{enrolmentId}/assessments	CM · CLN	ASSIGN_ASSESSMENT	Assessment·assign	require	
	POST	/assessments/{assessmentId}:complete	CM · CLN	COMPLETE_ASSESSMENT	Assessment·complete	require	AssessmentCompleted (+ RiskClassificationChanged)
careplan	GET	/care-plans/{carePlanId}	CM · CLN	VIEW_CARE_PLAN	CarePlan·get	require	
	POST	/care-plans	CM	CREATE_CARE_PLAN	CarePlan·create	require	
	POST	/care-plans/{carePlanId}:submit	CM	SUBMIT_FOR_APPROVAL	CarePlan·submit	require	SubmittedEvent → TaskAssigned (PLAN_APPROVAL)
	POST	/care-plans/{carePlanId}:activate	CLN	APPROVE_CARE_PLAN	CarePlan·activate	require	CarePlanActivated · TaskCompleted · TaskAssigned ×n
	POST	/care-plans/{carePlanId}:request-changes	CLN	REQUEST_PLAN_CHANGES	CarePlan·requestChanges		DecidedEvent → TaskCompleted
	POST	/care-plans/{carePlanId}:reject	CLN	REJECT_CARE_PLAN	CarePlan·reject		DecidedEvent → TaskCompleted
	POST	/care-plans/{carePlanId}:revise	CM	REVISE_CARE_PLAN	CarePlan·revise	require	CarePlanRevised
carework	GET	/work-queues/{role}	CM · CLN · PA · PC · PLA	VIEW_WORK_QUEUE	Task·workQueue		
	POST	/tasks	CM · PA	ASSIGN_TASK	Task·create	require	TaskAssigned
	POST	/tasks/{taskId}:complete	CM · CLN	COMPLETE_TASK	Task·complete	require*	TaskCompleted
referral	GET	/referrals	CM · CLN · PC	VIEW_REFERRALS	Referral·list		
	POST	/referrals	CM · CLN	CREATE_REFERRAL	Referral·create	require	ReferralCreated
	POST	/referrals/{referralId}:record-outcome	CM · CLN · PC	RECORD_REFERRAL_OUTCOME	Referral·recordOutcome	require*	ReferralAccepted ReferralCompleted
	POST	/provider-participations	PA	REGISTER_PROVIDER_PARTICIPATION	Referral·registerParticipation		
	POST	/provider-actions	PC · CLN	RECORD_PROVIDER_ACTION	Referral·recordProviderAction		ProviderActionRecorded
medication	POST	/refill-requests	CM	CREATE_REFILL_REQUEST	Refill·create	require	RefillRequested
	POST	/refill-requests/{refillRequestId}:record-fulfilment	CM · PC	RECORD_FULFILMENT	Refill·recordFulfilment	require	RefillFulfilled
observation	POST	/observations	CM · CLN · PC	CAPTURE_OBSERVATION	Observation·capture	require	ObservationCaptured (+ ThresholdReviewRequired, TaskAssigned)
	POST	/observations/{observationId}:invalidate	CM · CLN	INVALIDATE_OBSERVATION	Observation·invalidate		
caregap	GET	/enrolments/{enrolmentId}/observations	CM · CLN	VIEW_OBSERVATION_TREND	Observation·trend	require	
	GET	/care-gaps	CM · CLN · PA	VIEW_CARE_GAPS	CareGap·list		
	POST	/care-gaps	CM · PA	DETECT_CARE_GAP	CareGap·record	require	CareGapDetected
	POST	/care-gaps/{careGapId}:suppress	CM · CLN · PA	SUPPRESS_CARE_GAP	CareGap·suppress		
engagement	POST	/care-gaps/{careGapId}:close	CM · CLN · PA	CLOSE_CARE_GAP	CareGap·close		CareGapClosed
	POST	/outreach-requests	CM	REQUEST_OUTREACH	Outreach·request	require*	OutreachRequested
	POST	/outcomes:record	PA	RECORD_OUTCOME	Outcome·record	require*	OutcomeRecorded
outcome	GET	/programmes/{programmeId}:proof	PA · PM · ES	VIEW_PROGRAMME_PROOF	Outcome·proof		writes zc_report_snapshot
audit	GET	/audit	PLA · PA	AUDIT_QUERY	Audit·query		

Roles: CLN clinician · CM care manager · PA programme admin · PC provider coordinator · PM payer manager · ES employer sponsor · PLA platform admin (@PreAuthorize; MEMBER has no endpoint). Every POST needs Idempotency-Key.
Consent gate: require = ConsentGate.require(enrolment, HEALTH_CONTENT), throws CONSENT_REQUIRED / CONSENT_REVOKED; require* = conditional (SAFE_EXIT tasks exempt; cancelled/declined referrals; only health-content outreach; only enrolment-level outcomes); status = read only.
In-handler role rules beyond @PreAuthorize: plan author cannot approve; work queue only for a role held (or PLA); provider users act only for their own organisation; proof audience by role (PA all, PM payer, ES employer_aggregate).

Error model & database enforcement

Exception codes, how failures become responses, and the rules PostgreSQL enforces itself

ZCAREEXCEPTIONCODE REGISTRY (24 CODES)

Code	Severity	escalateTo	nextActions	How	Raised by
ZCARE_CONSENT_REQUIRED	ESCALATE	CARE_MANAGER	request_consent	throw · warn	ConsentGateService.require; EnrolmentService.activate; captureConsent, care-context (warn)
ZCARE_CONSENT_REVOKED	HARD_STOP	–	[]	throw · warn	ConsentGateService.require; OutreachService.request; care-context read (warning)
ZCARE_ENROLMENT_DUPLICATE	WARNING	–	proceed	throw · db	EnrolmentService.invite; uq_zc_enrolment_one_active
ZCARE_GAP_DUPLICATE	INFO	–	proceed	throw · db	CareGapService.record; uq_zc_care_gap_dedup
ZCARE_GAP_SUPPRESS_REASON_REQUIRED	WARNING	–	retry	throw · db	CareGapService.suppress; ck_zc_care_gap_action_reason
ZCARE_PROGRAMME_VERSION_IMMUTABLE	HARD_STOP	–	[]	throw · db	ProgrammeService.requireDraft; trg_zc_programme_version_immutable (+ template)
ZCARE_EXTERNAL_DEPENDENCY_UNAVAILABLE	WARNING	–	retry	warn	CohortService.identify (PayerUnavailableException)
ZCARE_PLAN_VERSION_IMMUTABLE	HARD_STOP	–	[]	db	trg_zc_care_plan_approval_immutable only
ZCARE_CLINICAL_APPROVAL_REQUIRED	HARD_STOP	–	[]	throw · db	ProgrammeService.publish; ck_zc_programme_version_publication,published_approved
ZCARE_CONFIG_HISTORY_IMMUTABLE	HARD_STOP	–	[]	db	trg_zc_config_history_append_only only
ZCARE_TASK_RESTRICTED	ESCALATE	ROLE_OWNER	escalate	throw	TaskService.complete (THRESHOLD_REVIEW is clinician-only)
ZCARE_GAP_ACTION_RESTRICTED	ESCALATE	ROLE_OWNER	escalate	throw	CareGapService.suppress / close (CM or PA only, OD-21)
ZCARE_COHORT_REVIEW_REQUIRED	ESCALATE	PROGRAMME_ADMIN	review	warn	CohortService.identify (volume above max_plausible_volume)
ZCARE_FULFILMENT_SOURCE_REQUIRED	WARNING	–	retry	throw · db	RefillService.recordFulfilment; ck_zc_refill_confirmed_requires_authorised_source
ZCARE_REFILL_DUPLICATE	INFO	–	proceed	throw · db	RefillService.create; uq_zc_refill_one_open
ZCARE_OUTREACH_FREQUENCY_EXCEEDED	WARNING	–	proceed	warn	OutreachService.request (held, not dropped)
ZCARE_AGGREGATE_THRESHOLD_NOT_MET	WARNING	–	proceed	warn	ProofService.proof (employer population below threshold)
ZCARE_THRESHOLD_RULES_MISSING	WARNING	–	escalate	warn	ObservationService.capture (stored, not evaluated)
VALIDATION_FIELD_INVALID	WARNING	–	retry	throw · handler	BusinessException.invalid / unknown; bean validation; unmapped constraints
VALIDATION_FIELD_REQUIRED	WARNING	–	retry	throw · handler	BusinessException.required; @NotNull / @NotBlank / @NotEmpty; no Idempotency-Key
IDEMPOTENCY_KEY_CONFLICT	HARD_STOP	–	[]	filter	IdempotencyFilter (same key, different body)
ZCARE_THRESHOLD_REVIEW_REQUIRED	ESCALATE	CLINICIAN	escalate	warn	ObservationService.capture (review task raised) · pending in 04B
ZCARE_CONCURRENT_MODIFICATION	WARNING	–	retry	handler	GlobalExceptionHandler (row_version optimistic lock) · pending in 04B
ZCARE_INTERNAL_ERROR	HARD_STOP	–	[]	handler	GlobalExceptionHandler.unexpected → HTTP 500, status error · pending in 04B

How: throw = BusinessException (transaction rolls back) · warn = ServiceResult.warning (work commits, envelope says exception) · db = translated from a violated constraint or trigger · filter / handler = raised outside services.
Below the dashed line: in code but not yet in the 04B registry (OD-17). A HARD_STOP always has empty nextActions.

HOW A FAILURE BECOMES A RESPONSE

No / bad / expired token	401	no envelope: {"error": "unauthorized"} + WWW-Authenticate
Tenant mismatch or inactive tenant	403	{"error": "tenant_mismatch" "tenant_not_active"}
@PreAuthorize, AccessDeniedException, unknown route	403	{"error": "forbidden"} via SecurityExceptionAdvice
BusinessException from a service	200	status exception; transaction rolled back
ServiceResult.warning	200	status exception; work committed
Bean validation, bad JSON, type mismatch	200	status exception; one entry per field; [retry]
DataIntegrityViolationException	200	ConstraintViolationTranslator → mapped code
OptimisticLockingFailureException	200	ZCARE_CONCURRENT_MODIFICATION
Transport ErrorResponse (415, 406 ...)	as is	{"error": detail}, outside the envelope
Anything else	500	status error · ZCARE_INTERNAL_ERROR · correlation id
Replayed Idempotency-Key, same body	200	stored envelope + Idempotency-Replay: true

CONSTRAINTVIOLATIONTRANSLATOR.CODES

uq_zc_enrolment_one_active	→ ZCARE_ENROLMENT_DUPLICATE
uq_zc_care_gap_dedup	→ ZCARE_GAP_DUPLICATE
uq_zc_refill_one_open	→ ZCARE_REFILL_DUPLICATE
trg_zc_programme_version_immutable	→ ZCARE_PROGRAMME_VERSION_IMMUTABLE
trg_zc_assessment_template_version_immutable	→ ZCARE_PROGRAMME_VERSION_IMMUTABLE
ck_zc_programme_version_publication	→ ZCARE_CLINICAL_APPROVAL_REQUIRED
ck_zc_programme_version_published_approved	→ ZCARE_CLINICAL_APPROVAL_REQUIRED
trg_zc_care_plan_approval_immutable	→ ZCARE_PLAN_VERSION_IMMUTABLE
trg_zc_config_history_append_only	→ ZCARE_CONFIG_HISTORY_IMMUTABLE
ck_zc_refill_confirmed_requires_authorised_source	→ ZCARE_FULFILMENT_SOURCE_REQUIRED
ck_zc_care_gap_action_reason	→ ZCARE_GAP_SUPPRESS_REASON_REQUIRED

Constraint names are API contract: renaming one changes the code a caller sees (ADR-0003).

RULES POSTGRESQL ENFORCES ITSELF (CONNECTED AS ZC_APP)

Row-level security · 56 policies

Every tenant table: ENABLE + FORCE ROW LEVEL SECURITY.
CREATE POLICY rls_<table>_tenant ON <table>
 USING (tenant_id = zc_current_tenant())
 WITH CHECK (tenant_id = zc_current_tenant());
zc_current_tenant() = NULLIF(current_setting('zcare.tenant_id', true), '')::bigint
Exempt: zc_tenant, zc_tenant_whatsapp, zc_user, zc_security_event (ApplicationRolePostureIT).

Triggers · 77 on 60 tables

51 × zc_touch_row(): bump row_version, set updated_at, refuse id / tenant_id / created_* changes.
9 append-only tables × 2: *_append_only + *_no_truncate (zc_forbid_change).
Pins: trg_zc_enrolment_pins, _assessment_pins, _care_plan_pins (zc_forbid_pinned_change).
Frozen: programme_version, assessment_template_version (zc_forbid_published_version_change);
 care_plan_approval (zc_care_plan_approval_immutable).
trg_zc_task_dependency_gate · trg_zc_observation_correction_only (only the invalidation columns change).

One-live rules · partial unique indexes

uq_zc_enrolment_one_active (member, programme)
 WHERE status IN (invited, active, suspended)
uq_zc_consent_record_in_force (enrolment)
 WHERE status IN (captured, validated, scope_reduced)
uq_zc_care_plan_one_active (enrolment) WHERE active
uq_zc_refill_one_open (medication_coordination)
uq_zc_care_gap_dedup (member, version, type, period)
uq_zc_provider_participation_live · uq_zc_monitoring_schedule_active · uq_zc_role_assignment_no_overlap (EXCLUDE USING gist on validity period)

Keys, types and privileges

Ids: bigint GENERATED ALWAYS AS IDENTITY.
Composite FKs (tenant_id, id[, programme_version_id]) backed by uq_*_fk_target unique constraints, so a child row cannot cross tenants or versions.
Named CHECKs: status sets, reasons, sources (ck_*).
zc_app: LOGIN, NOBYPASSRLS, owns nothing, no DELETE, no UPDATE on append-only tables.
Hibernate ddl-auto = validate; Liquibase (schema owner) is the only writer of DDL.

HOW THE BUILD PROVES IT (SRC/TEST/JAVA/.../DB, THE ONLY TESTS TODAY)

ChangelogConventionsTest

unit · 7 tests

Changelog file rules without a database: includes-only master, file names and order, changeset ids, headers, HALT preconditions, rollbacks, vendor SQL tagged.

ChangelogMigrationIT

IT · 3 tests

Applies the changelog to an empty PostgreSQL 16 as a non-superuser schema owner; re-running it is safe.

AgreedDesignParityIT

IT · 1 test

Builds the changelog and the agreed design (db.sql with the real V003) side by side; every object, privilege and comment must match.

ApplicationRolePostureIT

IT · 6 tests

zc_app limits, RLS on every tenant table, tenant_id leading every index, explicit constraint names; the four RLS-exempt tables are listed.

TenantIsolationIT

IT · 7 tests

Runs as zc_app: one tenant can never read or write another tenant's rows.

Not yet tested: services, controllers, filters and the ArchUnit package rules (dependency present, no rule classes).